1. What is fragmentation? How is fragmentation handled by operating system.

In an operating system, **fragmentation** is a memory management problem that occurs when free memory space is broken into small pieces (fragments).

Even if the total free memory is enough to satisfy a process request, the process may not get allocated if free memory is not contiguous.

Fragmentation means **wastage of memory** because the memory is not organized properly.

Types of Fragmentation :-

1. Internal Fragmentation:

- Occurs when allocated memory is larger than requested.
- Extra unused space inside a block is wasted.

Example:

- Suppose memory allocation works in **blocks of 4 KB**.
- A program requests 6 KB → it gets 2 blocks (8 KB)
- Actual use = 6KB, unused = 2 KB Wasted (Internal fragmentation)

2. External Fragmentation:

- occurs when free memory is scattered into many small non-contiguous blocks.
- Even through total free memory is enough, a process cannot be allocated because it needs **contiguous memory**.
- Example:

  suppose free spaces = 2 KB + 3KB + 4 KB (total 9KB free)

  A process of 8KB cannot be loaded because no single block is large enough → External fragmentation.

Handling Internal Fragmentation:

- use **smaller fixed partitions** (but this increases overhead)
- use **dynamic memory allocation** (Exact memory allocated as per requirement)
- Example: Paging helps avoid internal fragmentation because memory is allocated in **fixed-size** pages and processes use them efficiently.

Handling External Fragmentation:

Compaction → OS moves processes together to create one big free space. (But compaction is time-consuming and costly)

Paging → Break process into small pages and put them in any free memory slots. (No need for continuous space).

Segmentation → reduces external fragmentation by dividing processes logically (like code, data, stack).

Dynamic Storage Allocation Algorithms:

- **First Fit**: Take the first free space that fits
- **Best Fit**: Take the smallest free space that fits.
- **Worst Fit**: Take the largest free space.

- Internal fragmentation: OS reduces it using paging @ dynamic allocation.

- External fragmentation: OS reduces it using compaction, paging, segmentation and allocation strategies.

3. Explain FCFS, SSTF, SCAN and C-SCAN disk scheduling algorithms with examples.

Disk Scheduling Algorithms are needed because a process can make multiple I/O requests and multiple processes run at the same time.

The requests made by a process may be located at different sectors on different tracks. due to this, the seek time may increase more. These algorithms helps in minimizing the seek time by ordering the requests made by the processes.

Important Terms

Seek Time: It is the time taken by the disk arm to locate the desired track.

Rotational Latency: the time taken by a desired sector of the disk to rotate itself to the position where it can access the Read/Write heads is called **Rotional latency**

Transfer Time: It is the time taken to transfer the data requested by the **processes.**

Disk Access Time = Seek Time + Rotational Latency + Transfer time

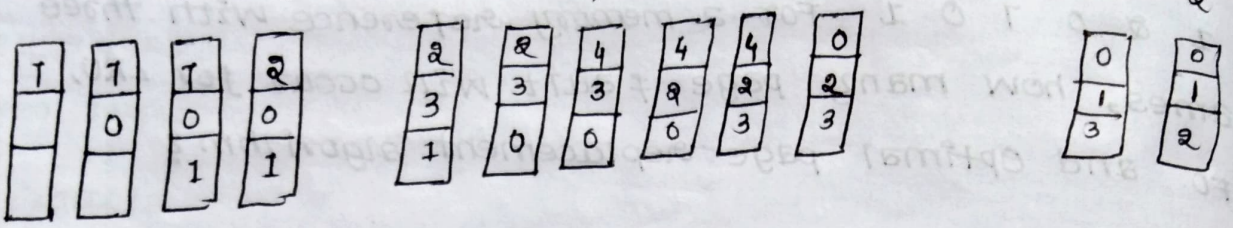
1. First Come First Serve (FCFS):

In this algorithm, the requests are served in the order they come. Those who come first are served first.

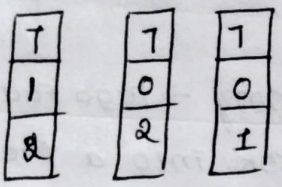
Example: Suppose the order of requests are 70, 140, 50, 125, 30, 25, 160 and initial position of the Read-Write head is 60.

FIFO (First in First Out) : Remove the page that entered memory first.

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2



0 1 7 0 1

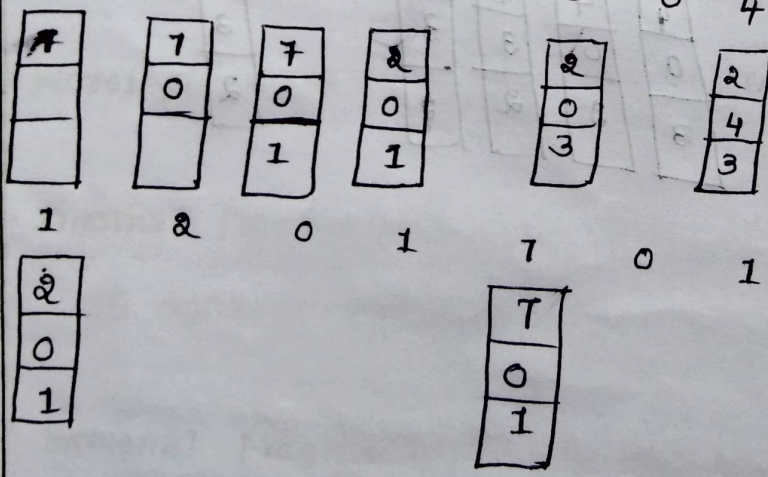


Total page Fault = 15

Optimal Algorithm :

- Replace the page that will not be used for the longest time in the future.
- this gives the least number of page faults (but not practical, since future use is unknown).

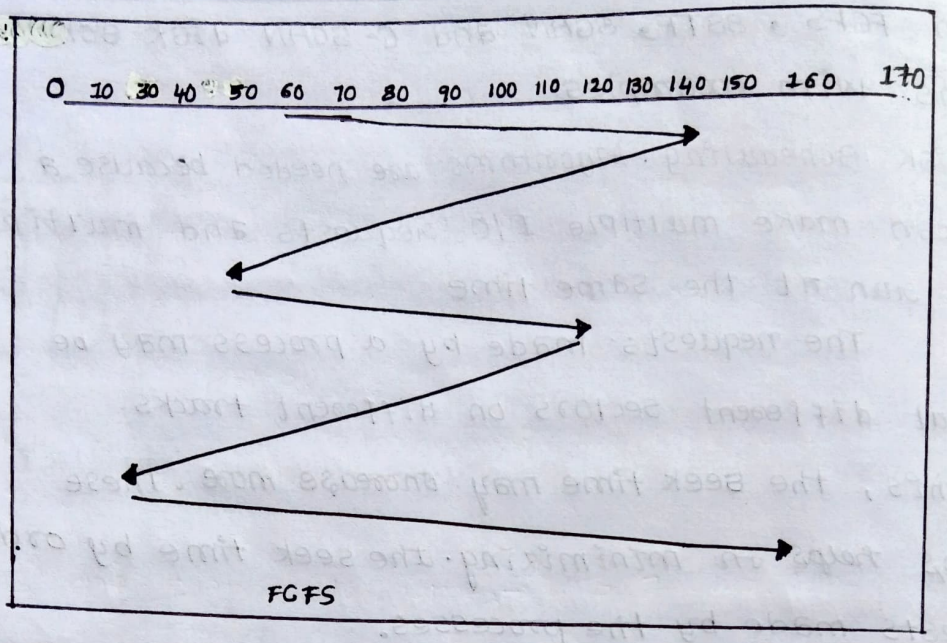
7 0 1 2 0 3 0 4 2 3 0 3 2



1 2 0 1 7 0 1

Total page Fault = 9

Example:



seek time = Distance moved by the disk arm:

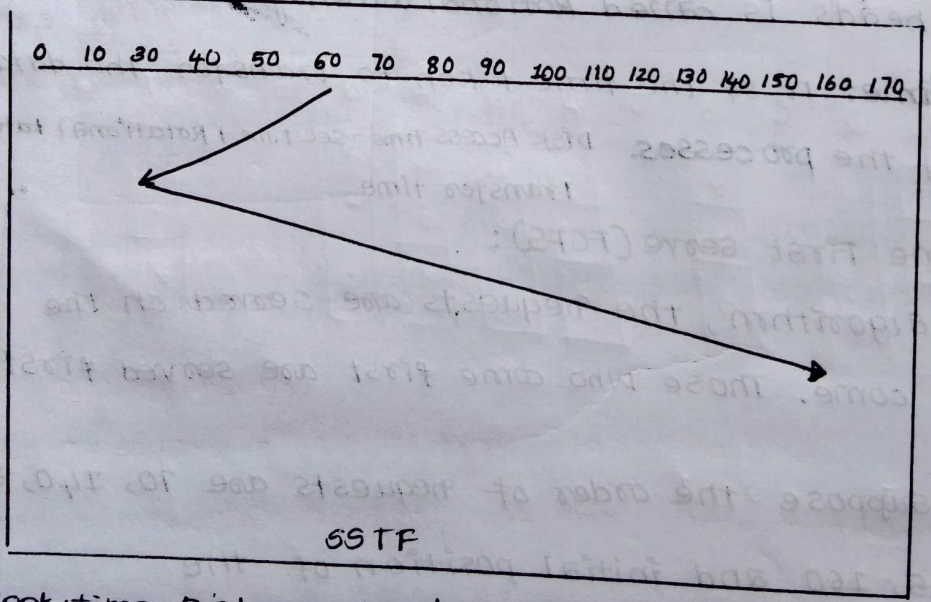
$= (140-70)+(140-50)+(125-50)+(125-30)+(30-25)+(160-25) = 480$

Cons:

- Does not try to optimize seek time:
  - may not provide the best possible service.

2. Shortest seek time First (SSTF):

EX Always pick the request nearest the current head.



Seek time = Distance moved by the disk arm

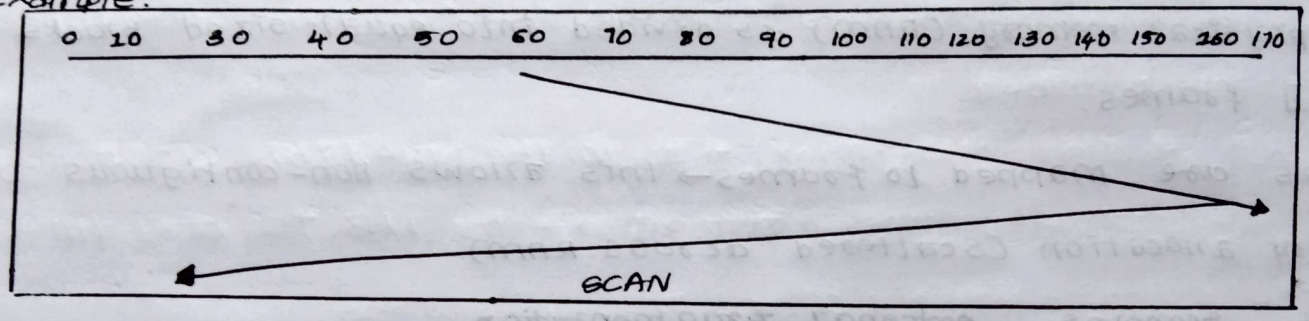
$= (60-50)+(50-30)+(30-25)+(70-25)+(125-70)+(140-125)+$

$(160-125) = 270$ //.

3. SCAN (Elevator Algorithm):

Assume head initially moves toward higher-numbered cylinders. It serves requests in that direction, goes to the end (cylinder 199), the reverses and serves remaining requests on the way back. Head moves in one direction serving requests until the end.

Example:



Seek time = Distance moved by the disk arm.

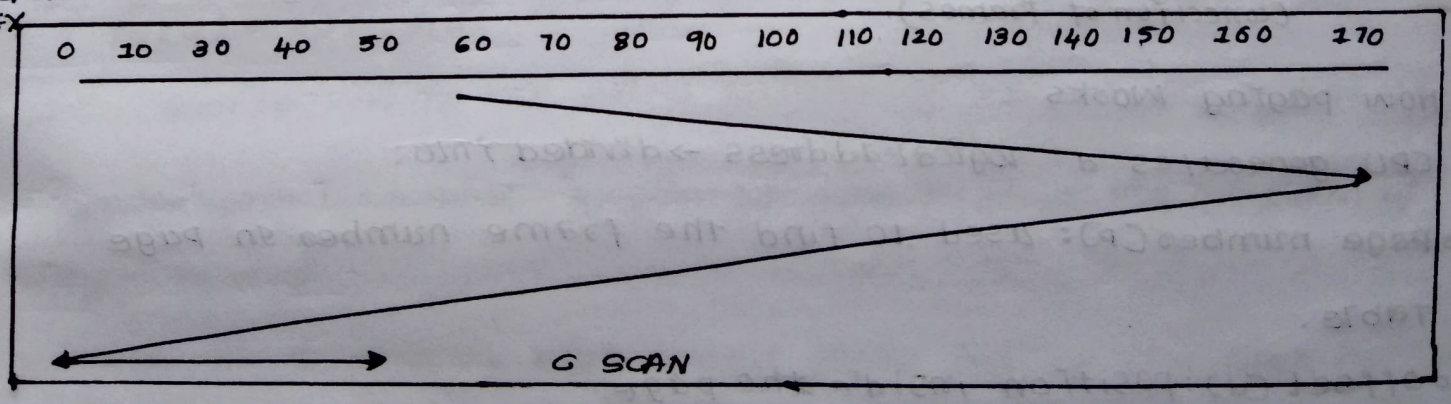
$= (170-60) + (170-25) = 255$

Cons: Long waiting time for requests for locations just visited by disk arm.

4. C-SCAN: (Circular SCAN)

Head moves only in one direction (up), serving requests until the end, the jumps to cylinder 0 and continues moving up. The jump from end to start is usually considered an instantaneous reposition (no counted head movement).

Ex



Seek time = Distance moved by the disk arm

$= (170-60) + (170-0) + (50-0) = 330$

Physical address = Frame number + Offset

Data is fetched from memory

Example : ∘ Page size = 1KB

∘ Logical address = 2049

∘ page number = 2049 ÷ 1024 = 2

∘ offset = 2049 % 1024 = 1

If page 2 is stored in frame 5 → physical Address = (5X1024) + 1 = 5121.

Problem in simple paging.

∘. First, we must read the page table (store in main memory).

∘ Then we must access the actual data from memory.

∘ So, two memory accesses are required instead of one.

∘ This doubles the effective access time.

Example :

∘ If normal memory access = 100 ns,

∘ With paging

  ∘ page Table access = 100 ns.

  ∘ Data access = 100 ns

  ∘ total = 200 ns.

This slowdown is the main problem of simple paging (use TLB for overcome).

Translation Lookaside Buffer is used to solve the problem of simple paging.

∘ TLB is a small, high-speed cache inside the CPU.

It stores recently used page table entries (page-number → frame number).

4. With a neat diagram explain paging technique and analyse the problem in simple paging technique. Explain how TLB is used solve the problem.

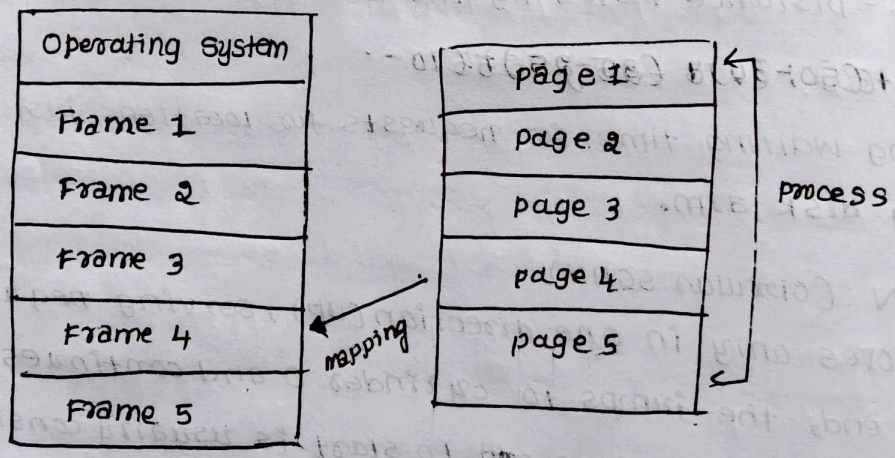
Paging is a memory management technique in which:

the logical memory (process) is divided into equal-sized blocks called pages.

The physical memory (RAM) is divided into equal-sized blocks called frames.

pages are mapped to frames → this allows non-contiguous memory allocation (scattered across RAM)

This removes external fragmentation.



main memory

(connection of frames).

How paging works:

CPU generates a logical address → divided into:

- Page number (p): used to find the frame number in page table.
- Offset (d): position inside the page
- page number is looked up in the page table to find the corresponding frame.

- Accessing the TLB is much faster than accessing the page table in main memory.

TLB WORKS:

- CPU generates a logical address
  - divide into page number (p) and offset (d)
- check TLB first
  - If the page number is found in TLB (TLB Hit) - directly get the frame number.
  - If not found (TLB-miss) → go to the page table in memory, fetch the frame number, and update the TLB for next time.
- combine Frame number + Offset physical address.
- Access the actual data in RAM.

Example: Assume memory access time = 100 ns

Assume TLB lookup time = 10 ns

Assume TLB Hit Ratio = 90%. (90% of the time we find the entry in TLB)

Case 1: TLB Hit (90% of the time)

Time = TLB access (10 ns) + Memory access (100 ns) = 110 ns.

Case 2: TLB Miss (10% of the time)

- time = TLB access (10 ns) + Page Table access (100 ns) + Memory access (100 ns) = 210 ns.

Effective Access Time (EAT):

$= (0.9 \times 110) + (0.1 \times 210)$

$= 99 + 21$

$= 120$ ns

Compared to simple paging (200 ns), using TLB makes it much faster.

5. a. Describe allocation of free frames.

- In paging (memory management), physical memory (RAM) is divided into fixed-size blocks called frames.
- Each process is divided into pages, and these pages are loaded into available frames.
- But since memory is limited, the Operating System (OS) must decide **how many** to give each process.

This decision process is called **frame allocation.**

Why is Free Frame Allocation Important?

If a process gets to few frames, it will cause **frequent page faults** (because required pages are not in memory)

- If a process gets too **many frames**, other processes may starve for memory.
- So, the OS must balance frame allocation fairly and efficiently

Allocation of free frames.

Two approaches:

1. Equal Allocation:

Each process gets the **same number of frames**, regardless of its **size.**

Example: suppose total free frames = 12, number of processes = 3

Each process gets $12 \div 3 = 4$ frames each.

Simple & fair but may waste memory because small processes may not need so many frames.

2. Proportional Allocation:

- Frames are allocated based on process size.
- Larger processes get more frames, smaller process get fewer.

Formula:

Frames given to process i = (size of process i / Total size of all processes) x Total number of frames.

Example:

- Total free frames = 12

Process P1 size = 20 KB, P2 = 10 KB, P3 = 30 KB (Total = 60 KB).

P1 gets (20/60) x 12 = 4 frames.

P2 gets (10/60) x 12 = 2 Frames

P3 gets (30/60) x 12 = 6 Frames.

This method is more efficient because it respects process size.

3 priority allocation: (Advanced method)

- Some processes are more important (high priority)
- This reduces their page faults and increases system performance.

4. Global vs Local Allocation:

Global Allocation: A process can take frames from other processes when needed. [Improves - Flexibility]

Local Allocation: A process can use only the frames allocated to it, cannot borrow from others. [Improves - Stability]

(b) What is thrashing ? How to Overcome thrashing.

Thrashing happens when a process does not have enough frames to run smoothly.

The process keeps generating page faults because the required page are not in memory. [Page-fault rate is very high].

CPU Spends more time swapping pages in and out of memory instead of executing instructions.

This situation where the system spends most of its time servcing page faults instead of actual work is called Thrashing

Why Trashing Happens:

1. too many processes in memory (high multiprogramming level)
   - Each process gets fewer frames.
   - They compete for memory, causing page faults.
2. Insufficient frame allocation:
   - processes are larger than the number of frames allocated.
3. poor page replacement policy:
   - Replacing useful pages frequently increases page faults.

Effects of Thrashing:

- CPU utilization decreases (CPU often idle waiting for memory
- Response time increase drastically.
- system peaformance goes down.

How to detect Thrashing: Monitor Page Fault Frequency (PFF)

- CPU utilization graph.

HOW to overcome Thrashing:

1) Working set model:

- Each process has a "working set" = set of pages it is actively using
- If enough frames are allocated to cover its working set few page faults.
- OS ensures each process gets at least its working set size.

2) Page Fault Frequency (PFF) control

- Monitor how frequently page faults occur:

- If too high → allocate more frames to the process.
- If too low → frames may be reduced and given to others.

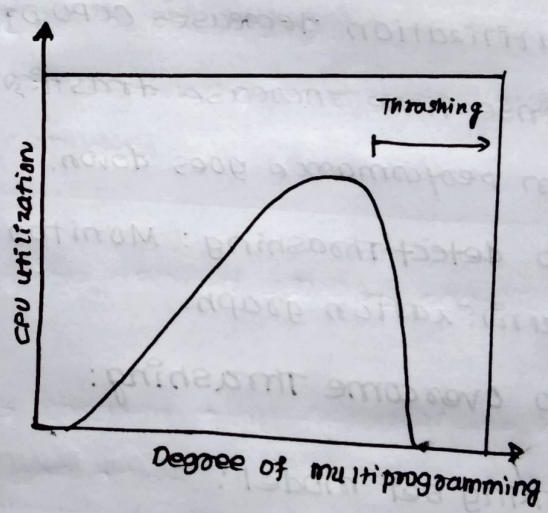
3) Reduce Multiprogramming:

- If system is overloaded with too many processes - suspend @ swap out some processes.
- This frees frames and reduces thrashing.

4) use Better Page Replacement Policies:

Like LRU (Least Recently Used) @ Priority Replacement to avoid replacing using pages.

Example: suppose Memory has 12 frames 4 processes are running, each needing at least 4 frames to execute properly Total need = 16 frames, but only 12 available.



Each process gets fewer frames than required → every process keeps replacing pages frequently thrashing occurs.

In the OS reduces multiprogramming to 3 processes, each process now gets 4 frames → Thrashing stops.

6. a. What is protection? Discuss principles of protection.

- protection in operating system means controlling the access of processes, users, and programs to the resources of the computer system (like CPU, memory, files, devices).
- Its goal is a prevent unauthorized access & avoid interference between processes.

protection = deciding who can use what, and how they can use it.

Principles of Protection :

- Principle of Least Privilege:

A process or user should be given only the minimum access rights needed to perform its task.

Example: A student should have read-only access to exam papers, not write access.

- Need-to-know Principle:

Access should be granted only if it is necessary for the work.

Example: A payroll program needs access to salary data but not to student exams records.

- Separation of Privilege:
- Multiple conditions may be required to access a resource.

Example:

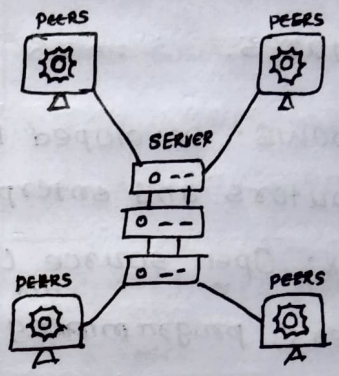
To withdraw money online - you need both password & OTP.

- Least common mechanism:
- Avoid sharing mechanisms between users/processes as much as possible.

a. Types of NOS

- Peer to Peer NOS
  - In this system; all computers are equal (peers).
  - Each computer can act as both client and server.
  - No central control → users share files, printers, @ internet directly.

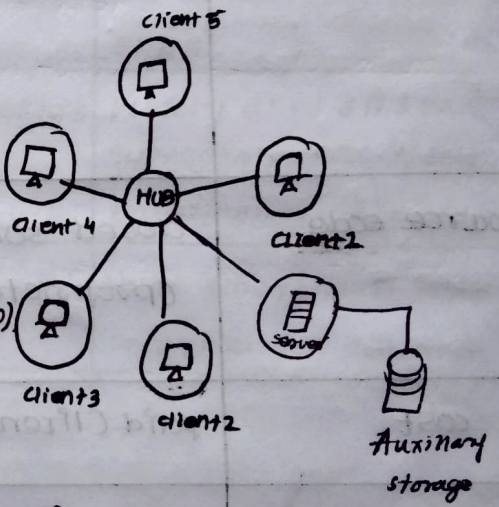
  Example: Windows 95/98, simple file sharing in Windows.



Advantages: ○ simple and INEXpensive ○ Easy to setup.

Cons: Poor security ○ Not Scalable for large network.

- Client - Server NOS:
  - A dedicated central server manages resources.
  - clients (other computers) request services (file, printing, authentication)
  - the server enforces security and permissions.



Example: Windows Server OS (2016, 2019)

- UNIX / Linux server distributions (Red Hat, Ubuntu Server)

Advantages:

- Better security and centralized control.
- Scalable to large networks
- Easier backup and management.

Disadvantages:

- more expensive
- Requires trained administrators.

- Because shared mechanisms can become a way for one user to attack another.

- **Open Design:**
  - Protection should **not depend** on secrecy of design.
  - Even if people know how the system works, it should still remain secure.

Example: Banking systems use encryption algorithms (like AES) that are publicly known, but still secure because of strong keys.

- **User Authentication and Access control:**
  - OS should check who the user is (authentication) & what operation are allowed (authorization).

Example: Logging into gmail → password identifies you, & permission decide what you can do.

- **Fail-safe Defaults:**
  - By default, access should be denied unless explicitly allowed.

Example: If a new user is created, by default he has no access until permission are given.

(b) Discuss about different types of Network Based Operating Systems.

A Network Operating System (NOS) is an OS that manages network resource like files, printers, applications and users across multiple computers connected in a network.

- Unlike a normal standalone OS, and NOS allows users to share resources and communicate over LAN, WAN, @ the Internet.

7 Compare Windows with Linux Operating System.

Both Windows and Linux are popular operating systems but they differ in design, usage, licensing and features.

Windows: Developed by Microsoft, mainly used in personal computers and enterprises.

Linux: Open source OS based on UNIX, Widely used in servers, programming and security systems.

FEATURE	Windows OS	LINUX OS.
Developer	Microsoft corporation	Open-source community (Linus Torvalds started it, with global contributions)
Source code	closed source (proprietary)	Open source (anyone can view, modify, & distribute)
cost	paid (license required)	Free (most distribution)
User interface.	GUI-based is default, easy for beginners	Mainly CLI (command Line interface) but modern distros (Ubuntu, fedora) provide GUI also
Security	More vulnerable to viruses and malware due to popularity	more secure, permissions-based, fewer viruses.

performance	Needs more resource (RAM, CPU)	Lightweight, runs even on old hardware.
customization:	Limited (restricted by Microsoft)	Highly customizable (open source, can modify kernel, desktop environment)
File system support	NTFS, FAT32, exFAT	ext2, ext3, ext4, Btrfs, XFS, etc.
software Availability	Large variety of commercial software (MS office, Adobe, etc)	Wide range of free/ open-source software but fewer commercial apps.
Usage Area	Commonly used in desktops laptops, business systems, and gaming	Widely used in servers, supercomputers, NW, programming, cybersecurity.
Community support.	Official Microsoft support online forms	Huge global open-source community support.

Example use cases:

Windows:

- offices use it for MS office, accounting software, ERP system.
- Gamers prefer Windows because most games are released for it.

Linux:

- used in servers (Google, Facebook, Amazon use Linux servers).
- preferred by programmers, system admins and cybersecurity experts.

8. What is access matrix ? How to secure OS using access matrix . Explain how is it implemented.

An Access Matrix is a security model used by Operating systems to define which users (subjects) can access which resources (objects) and in what way (might/ permissions).

Subjects : users, processes.

objects : files, memory , printers, Services.

Access rights : operations like read, write, execute, delete, print etc.

Access matrix ⇒ a table that shows Who can do what on which resource.

Example :

user / process	File 1	File 2	printer
user A	Read, Write	Read	Print
User B	Read	-	-
User C	-	Write	-

user A can read/write F1 and F2 print
user B can only read File 1
user C can only write to File2.

secure OS using access matrix

The OS uses the access matrix to check permissions whenever a process requests access to a resource.

- If the requested operation is allowed in the matrix - access is granted.
- If not present → access is denied.

Example : - If User B tries to write to File1
- OS checks matrix. User B has only "read" → deny request

This ensures confidentiality (only right people can see data), integrity (only right people can modify and availability (resource are not misused).

- problems of Access matrix: A real system has many users and resources → the access matrix becomes very large
- Storing the entire matrix is inefficient (lots of empty cells).
- solution : the access matrix is not stored as a whole but implemented using lists.

Implementation of Access Matrix :

1. Access Control List (ACL) : Column-wise implementation :-
   - For each object, store a list of subjects and their rights.
   - Example (for File 1):
     - File 1 → {User A : read, write; User B : read}.
   - Good for checking who can access a resource.

2. Capability list (C-List) → Row wise implementation :
   - For each subject, store a list of objects and allowed rights.
   - Example (for User A):
     - User A → {File 1 : read, write ; File2 : read, Printer : print}
   - Good for checking what a user can access.

Difference between ACL and capability list :

- ACL (Object-based): What operations are allowed on this object, and by whom?
- capability List (subject-based) : What operations can this user/ process perform, and on which objects.

9. a. Describe domain protection with the help of example.

Domain protection means ensuring that each process @ user can only access resource that belong to it's domain and nothing else.

This is impost for security, stability, and reliability of the system.

Without domain protection:

A student could delete admin files @ modify grades

With domain protection:

The OS prevents unauthorized access.

How it works:

- Each domain is associated with a set of access rights.
- Access rights specify which operations can be performed on a resource.

Example access right:-

- Read (R): Execute (X):
- Write (W): Delete (D):

Domain switching:

Sometimes a process may need to change its domain.

- Example: A student program may need temporary access to a printer. the OS switches it to a domain where "use printer" is allowed, and then switches back.

This switching is controlled by the OS to avoid misuse.

Domain protection ensures only permitted operations happen.

Example of Domain protection:

Imagine a file management system:

Resource	Student Domain	Admin Domain
Student Records	Read only (R)	Read + Write (R/W)
Exam papers	NO access	Read + Write (R/W)
Software Install	NO access	Execute + write (X/W)
printer	Execute (X)	execute (X)

- If a student tries to modify exam papers OS denies access (protection works).
- If admin modifies exam papers OS allows it.

(b) Distributed File system (DFS):

A Distributed File system is a method of storing and accessing file where the files are distributed across multiple computers (servers) but appear to users as if they are stored on a single local system.

It provides a unified, transparent view of files even though data is spread over different machines in a network.

Example: Google Drive files are not on laptop, but on different cloud servers. Still, you access them like they are on your own computer.

Key features:

1. Transparency:
   - users don't need to know where the file is physically stored.

Example: File might be on server in Delhi, but you access it as if it's on your PC

2. Resource sharing:
   - multiple users on different computers can share files.

3. Fault Tolerance:

Files are replicated on multiple servers if one server fails, file can still be accessed.

4. Scalability:

- New servers and storage can be added easily without affecting users.

6. Concurrency control:

Many users can access the same file, and DFS ensures consistency.

Architecture of DFS

1. **Client system**: Where user runs applications & requests files
2. **Server system**: Where actual files are stored and managed.

How it works (step by step):-

- A user requests to open a file (e.g., /home/project/report.txt)
- DFS software checks which server holds the file.
- File is fetched from that server and shown to the user.
- User reads/writes as if it stored locally.
- DFS ensures proper synchronization if multiple users access the same file.

Real-time Examples

- NFS - (Network File system)
- AFS - (Andrew File System)
- GFS - (Google File system)
- HDFS - (Hadoop Distributed File System)

Advantages of DFS

- Data sharing
- Reliability
- Scalability
- performance:

Disadvantages of DFS:

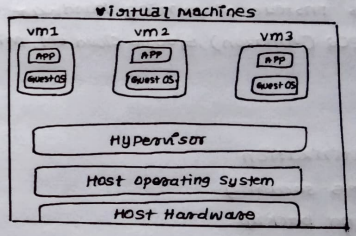
1. complexity
2. Network Dependency
3. Consistency Problems:

10. Discuss about Virtual Machines. Demonstrate its types and their implementations.

- A Virtual Machine (VM) is a software-based simulation of computer system.
- Virtual Machine (VM) is like a computer inside a computer.
- It is not physical hardware but software that like a real computer.
- A VM has: Its own CPU (virtual), memory (RAM), Storage, Operating System (Guest OS).

Example: Windows Laptop, Install VM software inside VMware, you create a VM and install Linux.

We can run Windows + Linux together on the same laptop.



Types of virtual Machine

- System Virtual Machine:-
  - creates a complete virtual computer with CPU, memory, storage.
  - can run multiple OS on one machine

Ex: We can run Window + Linux + macOS on one Laptop.

- Process Virtual Machine: [Does not simulate full hardware]:
  - provides a special environment for running a single progra
  - Helps in running the program on any OS

Ex: Java Virtual Machine (JVM)

Implementations of Virtual Machines:

1. Software based Hypervisors:
   - Run on top of an operating system (host OS)
   - Manage VMs as applications.

   Example: VMWare Workstation, Oracle VirtualBox.

2. Hardware-assisted Virtualization:
   - Modern CPUs (Intel VT-x, AMD-V) have virtualization support built-in.
   - Allows efficient VM execution.
   - Examples: KVM (Kernal Based Virtual Machine), Microsoft Hyper-V

3. Cloud based Virtual Machine:
   - provided by cloud service providers
   - users rent VMs instead of owning hardware
   - Examples: AWS EC2 (Amazon), Google Cloud VM, Microsoft Azure VM.)

Pros:
- Better resource utilization
- Isolation improves security
- Easy migration and backup.
- Useful for development, testing, and training.

Cons:
- Slower than physical machines (because of overhead).
- Requires more memory and CPU power from host
- Management can be complex for large VM clusters.